# M2: Rebuilding the full L3 book

*itchbook walkthrough, milestone 2. How order events become a book with every resting order in queue order, which rules decide each update, and what a full NASDAQ day showed when the book was held to account.*

## What "L3" means and why the queue matters

Market data comes in levels of detail. Level 1 is the best bid and best ask. Level 2 is the total size at each price. Level 3 is every individual order at every price, in the order it will be filled. On NASDAQ, as on most exchanges, resting orders at the same price fill in time priority: whoever arrived first trades first. So an order's fate depends on its queue position, meaning how many shares are ahead of it at its price, and not just on its price. The first research study (M4) is about exactly that quantity. Market makers and execution algorithms have to estimate queue position from L2 data, because L2 only shows the total at each price. L3 tells us the true value. The book built here is the source of that truth, so it has to be right to the share and to the place in line.

## The data structure, and why it is deliberately simple

`ReferenceBook` keeps three things. First, a hash map from order reference number to the order's symbol, side, price and remaining shares. Second, for each symbol and side, an ordered map from price to a price level. The bid side is sorted highest first and the ask side lowest first, so the best price is always the first entry. Third, inside each level, a running total of shares and a linked list of the orders queued there, front first. Each order record also stores an iterator to its node in that list, so removing an order from the middle of a queue takes constant time. Symbols are indexed by the day's stock locate number, which is a small dense integer, so finding a symbol's book is a plain array lookup.

This is not the fast version. Ordered maps and linked lists allocate a node for every insert and scatter related data across memory, which is slow for the CPU cache. On the Air the full day took about 6.5 minutes, roughly 930 ns per message. That is intentional. M2's job is to be obviously correct and easy to check. M3 builds the fast engine, and its gate is to produce output identical to this one, event for event. Writing the slow, clear version first gives the fast version an oracle it can't argue with.

## The rules for each event

Every message that changes the book is applied under a written rule, and the independent Python reference applies the same rules from its own code.

**Add orders** (A, and F with an attribution tag) insert a new order at the tail of its price level, because it is the newest at that price.

**Executions** (E, and C "executed with price") reduce the order's shares. The order is removed when it reaches zero. Repeated executions against one order accumulate.

A C message is sent when the trade price differs from the order's displayed price, as can happen in the opening and closing crosses. It must not reprice the resting order. It only reduces the order.

**Cancels** (X) reduce shares and keep the order's place in the queue.

**Deletes** (D) remove the order.

**Replaces** (U) are the subtle case. They remove the original order and insert a new one under a new reference number. The message carries no side or symbol, so both are copied from the original. The new order goes to the tail of its level even if the price is unchanged, because it is a new order and loses time priority. The shares field is the new total, not a change.

Trades against hidden orders (P), cross prints (Q) and broken trades (B) never touch the displayed book. Treating them as book events would be a classic bug.

Anything that breaks these rules is counted and never thrown:

- an execute, cancel, delete or replace referring to an unknown order (an "orphan");
- a duplicate add;
- a replace whose new reference is already live;
- a side other than buy or sell;
- an execution or cancel larger than the order (clamped to the order's size);
- a message whose header symbol disagrees with the order's own symbol.

A real day should produce none of these, and the count is how you find out.

## Checking the book against something it can't fake

Three layers of tests check the book.

**Unit tests.** One per rule: time priority, partial cancels keeping their place, replaces losing priority, executions removing exhausted orders, and anomalies leaving the book unchanged.

**A property test against a brute-force model.** RapidCheck generates hundreds of random sequences of up to 400 events over a tiny universe: three symbols and seven prices, so levels are crowded and replaces collide. About a tenth of the events deliberately refer to orders that never existed, and many more to orders that are already gone. The model keeps a flat list of orders, each with an arrival counter.

After every single event it recomputes every price level from scratch by sorting and grouping. At the end it checks every live order's queue rank and shares ahead.

The model is too slow for real data, but it is so simple that it is hard to get wrong. Three bugs were deliberately planted in the book to check that the tests notice:

- forgetting to reduce a level's total on a partial fill;
- putting new orders at the front of the queue;
- removing orders only when they go below zero instead of at zero.

Each was caught.

**An independent reference over a full day.** `tools/itchref/bookref.py` rebuilds the whole market in Python. Its queues are insertion-ordered dictionaries, so dictionary order is time priority. Every 30 minutes from 03:30 to 20:30 both implementations print a checksum of the entire book. The checksum is a sum of hashes:

- over every price level, of (symbol, side, price, total shares, order count);
- over every live order, of (reference, symbol, side, price, shares, rank in queue).

Because ranks are included, the checksum detects a single order in the wrong place in a single queue anywhere in the market. Both implementations also print the top five levels of a handful of symbols, and the full list of counters.

The hash is SplitMix64, a standard 64-bit mixing function, implemented once in each language and checked against its published reference outputs. The Python book cross-check has its own mutation tests: re-queueing a replace in place instead of at the tail, a cancel that forgets the level total, and an execution larger than the order that goes uncounted. Each one breaks the match.

## The gate: 30 January 2020

The book processed all 423,285,709 messages of the day:

- 186,610,705 adds, 8,555,084 executions, 4,990,972 partial cancels, 180,285,101 deletes and 36,777,372 replaces. Each equals the parser's count of the corresponding message types.
- **Zero orphans and zero anomalies of any kind.** Every execute, cancel, delete and replace referred to an order that was live at that moment.
- The book peaked at 1,925,638 resting orders and ended the day completely empty, with zero orders and zero levels. Every order added during the day was accounted for by the time the session closed.
- As a sanity check, at noon Apple's best bid was $321.92 and its best ask $321.95, consistent with where it traded that day.

The independent Python book took 29 minutes and 1.1 GB of memory, against 6.5 minutes for the C++ reference. Its output was **byte-for-byte identical** to the C++ output: 2,298 lines, consisting of every counter, all 35 whole-market checksums and 2,240 top-of-book rows. Two programs written separately, in different languages and with different data structures, agree on the position of every one of up to 1.9 million resting orders, at every half hour of the trading day.

### Crossed books

A crossed book is one where the best bid is above the best ask. In continuous trading that shouldn't exist, because the two orders would have matched. Monitoring every update of every symbol found ten crossed episodes during the day. All ten began while the symbol was halted or paused (trading state H or P), when orders can be entered but nothing executes. In four cases the book was still crossed when trading resumed. Each was cleared within one millisecond of the resumption, as the halt cross executed. **No symbol's book ever became crossed during normal continuous trading.** That is the M2 gate, and it holds.

### Who gets filled: the first look at queue priority

Because the book knows each order's position, it can check whether each execution hit the first order in its queue. During regular hours, 99.87% of plain executions (E) hit the order at the head of its queue, exactly what strict time priority predicts. The remaining 0.13% (10,744 of 8.3 million) did not. The "executed with price" messages (C) behave very differently: 83% of them hit an order that was not first in line. That fits what C messages mostly are, allocations in the opening and closing crosses, where the auction does not fill strictly by arrival time. Both facts feed straight into M4. A queue model that assumes strict time priority is right for continuous trading and wrong for auctions. The 0.13% residual in continuous trading is the first thing M4 will explain, before relying on the assumption.

## What this costs, and what M3 must beat

At roughly 900 ns per message, about 75 ns of it decompression, the reference book is about seven times slower than CppTrader's published optimised figure of 120 ns, and it allocates on every insert. M3 replaces the ordered maps and linked lists with flat, cache-friendly structures: an open-addressing hash table for orders, price levels stored in arrays, and pooled memory with no allocation on the hot path. It must reproduce this book's checksums and counters exactly at every snapshot.

## Questions to expect

*Why does a replace lose priority?* Because NASDAQ treats it as a cancel plus a new order. It carries a new reference number, and every later event for it uses the new number. If replaces kept priority, the queue-position study would overstate how

quickly replaced orders fill.

*How can you be sure no order was misplaced in a queue, when only checksums are compared?* The checksum includes every order's rank within its level. For a misplaced order to go unnoticed, the 64-bit hash sums would have to collide by chance, which is astronomically unlikely, at 35 separate times of day. The mutation tests also show the comparison does fail when queue order is wrong.

*What does an empty book at the end of the day prove?* That every add was matched by events that removed exactly its shares. A missed message type, a wrong share field, or a mishandled replace would leave orders stranded, or produce orphans when their later events arrived. There were neither.

*Why measure crossed books by trading state?* Because a crossed book while halted is legitimate: orders accumulate and nothing executes until the reopening cross. A crossed book during continuous trading would mean the book is wrong. Separating the two turns "some books crossed" into a precise, checkable statement.